



AI Agents & Claude Skills: The Complete Deep-Dive Tutorial

Based on: "How AI Agents & Claude Skills Work (Clearly Explained)" — Greg Isenberg × Ras Mic

About This Document: This tutorial extracts, expands, and deepens every concept from the video. It goes beyond surface-level explanations to give you mental models, real-world frameworks, visual diagrams, code patterns, and actionable strategies for building AI agents that actually work.

Video Duration: 35:25 | **Views:** 454K+ | **Channel:** Greg Isenberg (644K subscribers)



Table of Contents

1. The Paradigm Shift: Models Are Now Good Enough
2. How AI Agents Actually Work: The Core Architecture
3. The Context Window: The Most Important Concept in AI
4. The System Prompt: Your Agent's DNA
5. Why Most Agent.md Files Are Unnecessary
6. What Are Skills? The Game-Changing Framework
7. Progressive Disclosure: How Skills Are Loaded
8. The Recursive Skill-Building Methodology
9. Real-World Skill Examples (Sponsor Vetting, Report Generation)
10. Sub-Agents vs. Single Agents: When to Scale
11. The AI Workflow Ladder: From Simple to Complex
12. Skill Marketplaces: The Coming Economy
13. The Permanent Underclass Warning & What To Do About It
14. Building Your Personal AI System: Action Plan

1. The Paradigm Shift: Models Are Now Good Enough

The most important and counter-intuitive insight from the video is this: **the bottleneck in AI productivity is no longer the model — it's the human's ability to structure work for the model.**

🔑 **Core Insight:** Ras Mic opens with a bold claim — *"The models are exceptionally good. Opus 4 is amazing. GPT-5 is amazing."* Most people are spending time trying to make the model smarter when the real opportunity is in building better workflows and context around an already-capable model.

The Two Camps of AI Users

Camp 1: Model Skeptics	Camp 2: Model Believers (The Right Camp)
Constantly switches models	Picks one model and builds deeply around it
Blames the model when output is bad	Improves context, skills, and prompts
Adds more instructions to agent.md	Strips down to minimal instructions
Tries to over-specify everything	Trusts the model's existing knowledge
Chases the newest model	Masters the current model deeply

What "Models Are Good" Actually Means

When Ras says the models are good, he means they have internalized massive amounts of domain knowledge. You don't need to tell Claude Code that "this codebase uses React" when it can read the codebase itself. Providing redundant instructions is like reminding a professional how to hold a pen every time they sit down to write.

Analogy: "If I'm building a website with Claude Code and I'm telling Claude, 'This codebase uses React,' I don't need to — because it has the codebase in context. It can check the code itself." — Ras Mic

Going Deeper: The Competence Spectrum

Modern frontier models (Claude 3.5+, GPT-4o, Gemini 1.5 Pro) operate at roughly the 85th-95th percentile of human competence in most knowledge domains. They don't need to be taught what a

React component is, what a marketing funnel looks like, or how to write a professional email. What they *do* need is:

- **Context** — What is the current situation?
- **Constraints** — What are the limits and rules?
- **Desired outcome** — What does "done" look like?
- **Your personal preferences** — How do YOU want it done?

Traditional (Wrong) Approach:

```
| USER → [Teach model basics + domain knowledge] |  
| [Write massive agent.md files] |  
| [Re-explain context every time] |  
| RESULT → Bloated context, worse performance |
```

Modern (Correct) Approach:

```
| USER → [Trust model's existing knowledge] |  
| [Provide specific context only] |  
| [Build skills for YOUR workflows] |  
| RESULT → Lean context, better performance |
```

2. How AI Agents Actually Work: The Core Architecture

Before diving into skills and optimizations, you need a crystal-clear mental model of what an AI agent actually is. This section builds that foundation from first principles.

The Three Levels of AI Interaction

LEVEL 1: LLM (Large Language Model)

You type → Model responds → Conversation ends (or continues)

One-way flow. No memory. No tools.

Example: Basic ChatGPT conversation

LEVEL 2: AI WORKFLOWS

Predefined sequences of AI steps

Human-designed pipeline, AI executes

Example: n8n automation, Zapier + AI steps

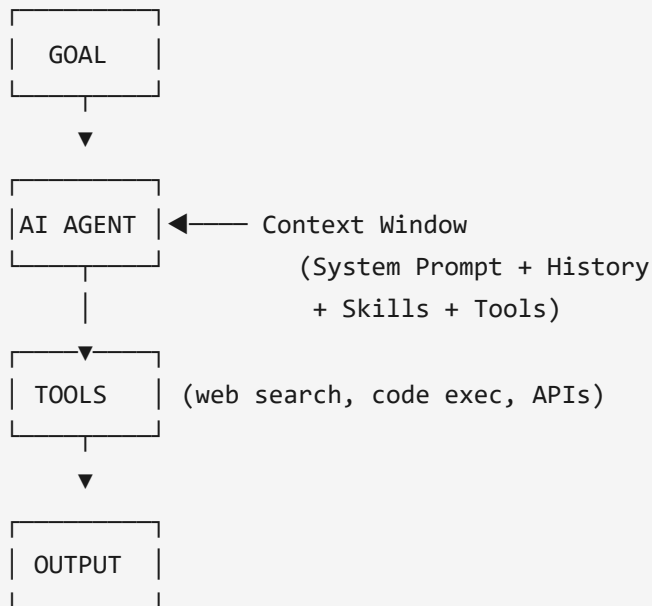
```
| | |
```



LEVEL 3: AI AGENTS

Goal-oriented. Self-directed. Tool-using.

The agent DECIDES what steps to take.



What Makes an Agent an "Agent"?

The defining characteristic of an AI agent vs. a simple chatbot is **autonomy over tool use and multi-step reasoning**. An agent can:

- Decide which tools to use and in what order
- Evaluate intermediate results and adjust its approach
- Break a complex goal into sub-tasks and execute them
- Loop back and retry if something fails
- Coordinate with other agents (sub-agents)

The Agent Execution Loop

The Agent Think-Act Loop:

1. OBSERVE → Read context window (what do I know?)
2. THINK → Plan the next action (what should I do?)
3. ACT → Execute tool call or produce output
4. OBSERVE → Read result of action
5. EVALUATE → Did this move me toward the goal?

6. REPEAT → Loop until goal achieved or stuck

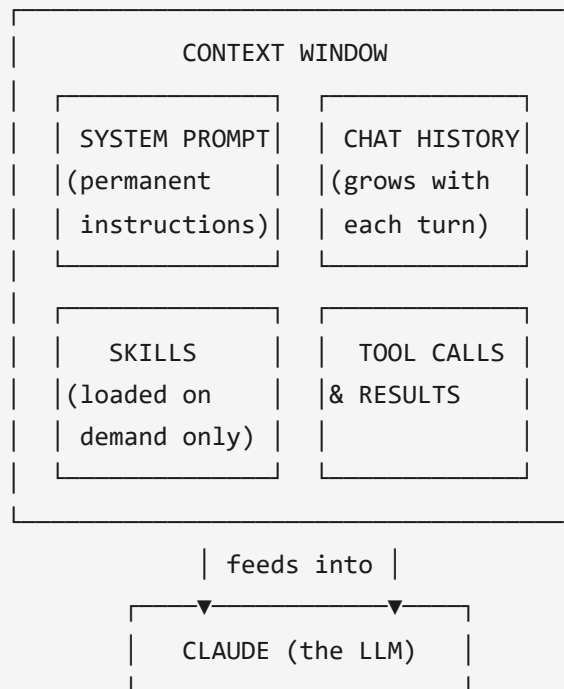
This is the "ReAct" (Reasoning + Acting) loop that most modern agentic frameworks implement under the hood.

Key Concept — The Agent Loop: Every time an agent takes an action (calls a tool, runs code, searches the web), the RESULT comes back into the context window. The agent then "reads" this result and decides what to do next. This is why context window size matters so much — each action and result adds to it.

Claude's Specific Architecture (Claude.ai & Claude Code)

When you use Claude as an agent, the architecture looks like this:

CLAUDE AGENT ARCHITECTURE:



3. The Context Window: The Most Critical Concept in AI

Ras Mic spends significant time on context windows because **most people fundamentally misunderstand how AI memory works**. This misunderstanding is the root cause of most agent failures.

💡 **Core Definition:** The context window is the AI's entire "working memory" — everything it can "see" and reason about in a single interaction. Once information falls out of the context window, the model has no access to it. It literally cannot remember it.

Context Window Size: What the Numbers Mean

Model	Context Window	Approx. Pages of Text	Real-World Equivalent
GPT-3.5	4K–16K tokens	3–12 pages	A short report
GPT-4o	128K tokens	~100 pages	A short book
Claude 3 Opus	200K tokens	~150 pages	A full novel
Claude 3.5 Sonnet	200K tokens	~150 pages	A full novel
Gemini 1.5 Pro	1M tokens	~750 pages	A full textbook

Note: 1 token ≈ 0.75 words in English. 1,000 tokens ≈ 750 words ≈ 1.5 pages.

What Lives Inside the Context Window

ANATOMY OF A CONTEXT WINDOW (what takes up space):

[PERMANENT – always loaded]

SYSTEM PROMPT	~500-2K tokens
(Your base instructions, personality,	
role definition, output format rules)	

[GROWS OVER TIME – accumulates each turn]

CONVERSATION HISTORY	VARIABLE
User: [message] → AI: [response]	
User: [message] → AI: [response]	
... (gets very long in long sessions)	

[ON-DEMAND – loaded when needed]

SKILLS (only when agent decides to use them)
--

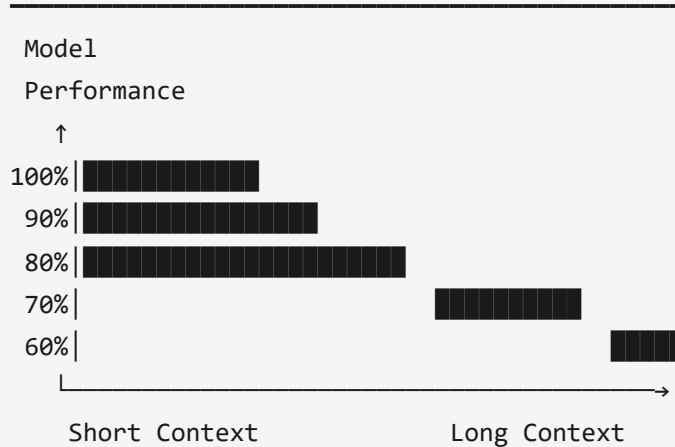
TOOL RESULTS (after each tool call)	
ATTACHED FILES (documents, code, etc.)	

Total $\leq$ Model's max context window size

The Context Window Efficiency Problem

Here's why context management is so important: **every token in the context costs money and slows the model down**. More importantly, studies show that models perform worse with very long contexts because they struggle to attend to all information equally (the "lost in the middle" problem).

CONTEXT WINDOW PERFORMANCE CURVE:



Key insight: Performance **DEGRADES** as context grows.
 The "lost in the middle" problem means info at the **BEGINNING** and **END** gets attended to much better than info buried in the middle.

The "New Employee" Mental Model

Ras Mic uses a brilliant analogy to explain what agents need. Imagine you hire a new employee. They're brilliant — top of their field. But on day one, they don't know **YOUR** business, **YOUR** preferences, or **YOUR** specific workflows. What do you give them?

- Not a general business textbook (they already know business)
- Not a definition of common terms (they know the vocabulary)
- But: **YOUR** specific processes, **YOUR** output format preferences, **YOUR** clients' contexts

The "Office Rundown" Analogy: In *The Office*, a new boss asks Jim for a "rundown." Jim spends the whole episode trying to figure out what "a rundown" means because it's company-specific jargon. AI agents are exactly like this new employee — brilliant in general, but confused about your specific, undefined workflow terms. **Your job is to define the specific things UNIQUE to your workflow, not explain general knowledge the model already has.**

Practical Context Management Strategies

Strategy	What It Is	When To Use
Fresh Sessions	Start a new chat for each task	For distinct, unrelated tasks
Minimal System Prompts	Keep permanent instructions lean	Always — avoid bloat
Skill-Based Loading	Load detailed instructions only when needed	Complex multi-purpose agents
Summarization	Have AI summarize long histories	Long-running sessions
Tool Outputs Pruning	Extract only relevant data from tool results	After web searches, API calls

4. The System Prompt: Your Agent's DNA

The system prompt is the most important configuration element of any AI agent. It defines the agent's identity, purpose, and behavior — and it's always in context (uses tokens on every single request).

 **Definition:** The system prompt is a special instruction block given to the model BEFORE any user conversation begins. It persists across the entire session. Think of it as the agent's "job description + personality + rules of engagement" permanently tattooed into its brain for that session.

Anatomy of an Effective System Prompt

SYSTEM PROMPT STRUCTURE (Minimal but Powerful):

[ROLE] — Who is this agent?

"You are a senior content strategist specializing in

B2B SaaS YouTube channels."

[CONTEXT] – What specific environment does it operate in?

"You work for [Company]. Our ICP is [description].

Our tone is [description]."

[BEHAVIORAL RULES] – What must it always/never do?

"Always provide sources. Never make up statistics.

Always ask for clarification before writing."

[OUTPUT FORMAT] – How should responses be structured?

"Responses should be in markdown. Use bullet points for lists of 3+ items. Keep under 500 words unless specifically asked for long-form."

[AVAILABLE SKILLS] – What tools/skills does it have?

(Usually handled automatically by the framework)

What SHOULD and SHOULD NOT Be in Your System Prompt

✅ INCLUDE in System Prompt	❌ DON'T INCLUDE in System Prompt
Your personal preferences for output format	General knowledge (model already knows)
Company/project specific context	Obvious instructions ("use professional tone" unless you've been getting otherwise)
Non-negotiable rules and constraints	Step-by-step procedures for every scenario (use skills instead)
How YOU define domain-specific terms	Everything you want the agent to "know" (if it's a workflow, make it a skill)
Tone and personality alignment	Redundant tech stack info (it can read the code)

The Minimalism Principle

Ras Mic advocates for **aggressive minimalism in system prompts**. The key insight is that a shorter, tighter system prompt actually produces better results because:

- Fewer tokens leaves more room for actual task context
- The model doesn't get confused by contradictory/redundant instructions
- Relevant instructions stand out more clearly when there's less noise

- Maintenance is easier — you're not maintaining a 5,000-word document

💡 **Practical Exercise:** Take your current system prompt. Remove every line that states something the model would "obviously know" or can figure out from context. Most people find they can cut 60-80% of their system prompt without any degradation in output quality.

5. Why Most Agent.md Files Are Unnecessary

One of the most provocative claims in the video: **"95% of people don't need an agent.md file."** This deserves a careful unpacking.

What Is an Agent.md File?

In the Claude ecosystem (and similar in Cursor's .cursorrules), an agent.md (or CLAUDE.md) file is a markdown file that sits in your project directory and is automatically loaded into the agent's context on every session. It's meant to give the agent persistent project-specific instructions.

WITHOUT AGENT.MD:

Session 1: System Prompt + Task
Session 2: System Prompt + Task
Session 3: System Prompt + Task

WITH AGENT.MD (Traditional Use):

Session 1: System Prompt + Agent.md + Task
Session 2: System Prompt + Agent.md + Task ← Same tokens wasted every time
Session 3: System Prompt + Agent.md + Task

WITH SKILLS (Ras Mic's Approach):

Session 1: System Prompt + Task + [Skill A loaded on demand]
Session 2: System Prompt + Task + [Skill B loaded on demand]
Session 3: System Prompt + Task + [No extra skills needed]
↑ Tokens only used when relevant!

The Core Argument Against Agent.md

The problem with agent.md is that it's a blunt instrument — it loads EVERYTHING, ALWAYS, regardless of whether it's relevant to the current task. Consider:

- You have a 500-line agent.md with 20 different workflow descriptions
- You ask the agent to "write a Python function to parse JSON"
- 90% of that agent.md is completely irrelevant to this task

- But the model has to process all 500 lines anyway
- This wastes tokens, adds latency, and can confuse the model with irrelevant context

Quote from Video: "If this was my agent.md file, imagine every single time — and we can actually check how many tokens this is — [my 116-line skill] would be 872 tokens loaded EVERY. SINGLE. TIME. whether I need it or not. With skills, it's only loaded when I actually need that specific workflow."

When Agent.md IS Appropriate

Despite the strong claim, there are legitimate use cases for agent.md files:

Use Case	Why Agent.md Works Here
Single-purpose agent	Every session uses the same context
Very short project instructions (< 200 tokens)	Negligible overhead
Stable project architecture info	Tech stack, file structure that never changes
Security or compliance rules	Must always be enforced, never forgotten
Team handoff context	New team members or agents need the same base context

 **The Rule of Thumb:** If your agent.md has more than 300 tokens and covers more than 2-3 use cases, you should probably be using Skills instead. Break it into skill files and let the agent load them on demand.

6. What Are Skills? The Game-Changing Framework

Skills are the central innovation that Ras Mic advocates for. Understanding skills deeply is the key to going from a "basic AI user" to someone who builds genuinely powerful, personalized AI systems.

 **Definition:** A Skill is a `.md` (markdown) file that contains a detailed, step-by-step procedure for a specific workflow you regularly perform. Unlike agent.md (always loaded), skills are **selectively loaded on-demand** — only when the agent determines they're relevant to the current task.

Skills vs. Agent.md vs. System Prompt: A Clear Comparison

Feature	System Prompt	Agent.md	Skills
When loaded	Always (every request)	Always (every session)	On-demand only
Token cost	Always pays tokens	Always pays tokens	Only pays when used
Purpose	Identity & base behavior	Project context	Specific procedures
Specificity	General/persona	Project-level	Task/workflow-level
Quantity	One per agent	One per project	Unlimited library
Best for	Personality, rules	Simple single-purpose	Multi-workflow agents

The Anatomy of a Skill File

SKILL FILE STRUCTURE (example: sponsor-vetting.md):

```
--- [METADATA SECTION – The Index Card] ---
name: Sponsor Vetting
description: Use when evaluating whether to accept a
            sponsorship offer. Checks credibility,
            fit, and past performance of potential
            sponsors via external research.

--- [PROCEDURE SECTION – The Instructions] ---
# Sponsor Vetting Procedure

## Step 1: Extract Company Info
Extract company name, website, and contact from email.

## Step 2: Research Phase
- Check their Twitter/X presence (follower count, engagement quality)
- Check their YouTube channel if applicable
- Check Trustpilot or G2 reviews
- Check Crunchbase for funding status
- Check if similar creators have worked with them

## Step 3: Rejection Criteria
Auto-reject if:
- 2+ of the above sources don't exist OR have poor standing
```

- Trustpilot score below 3.5
- No verifiable business history

Step 4: Output Format

Report to Google Sheets: [company, verdict, confidence, reason]

Why Skills Are So Powerful: Progressive Disclosure

The magic of skills is in how they're loaded. The agent doesn't load the entire skill file speculatively — it follows a **Progressive Disclosure** pattern:

PROGRESSIVE DISCLOSURE PATTERN:

STEP 1: Agent receives task

User: "Can you vet this sponsor email I forwarded?"

STEP 2: Agent reads ONLY the metadata (name + description)
of all available skills – just a few tokens each:

Skill Index (lightweight)	
• sponsor-vetting "Use when evaluating sponsorship offers"	← Agent reads this
• youtube-analytics "Use for channel performance reports"	
• linkedin-engine "Use when creating LinkedIn content"	

STEP 3: Agent recognizes "sponsor-vetting" matches the task

Agent thinks: "This task is about evaluating a sponsor.
The 'sponsor-vetting' skill is relevant. Load it."

STEP 4: Agent loads the FULL skill file (only this one)

Now the 116 lines / 872 tokens are loaded

STEP 5: Agent follows the skill procedure perfectly

TOTAL TOKENS USED: ~50 tokens (index) + 872 tokens (skill)
vs. Agent.md approach: 872 tokens EVERY. TIME.

Building Your Skill Library: Categories to Consider

Category	Example Skills
Content Creation	LinkedIn post generator, YouTube script writer, Newsletter template
Research & Analysis	Competitor analysis, Market research, Person research
Communication	Email response templates, Sponsorship vetting, Client follow-up
Business Operations	Meeting prep, Weekly reporting, KPI analysis
Technical	Code review, Documentation generator, Bug report template
Personal Productivity	Morning startup routine, End-of-day shutdown, Weekly review

🔑 **Real Example from the Video:** Ras Mic's actual skills include:

- **Sponsor Vetting Skill** — Automated research + scoring of sponsorship emails
- **YouTube Analytics Report Skill** — Pulls from 8 data sources, generates comprehensive channel report
- **Code Structure Skill** — 116-line specification for how to structure generated code for easy review
- **Morning Startup Sequence** — Daily initialization routine for his AI workflow

Each of these takes 10 minutes to execute but runs "flawlessly" after being built through the recursive methodology.

7. The Recursive Skill-Building Methodology

This is the most actionable framework in the entire video — and it's the thing most people skip. You can't just write a skill from scratch. You need to build it through iteration with the agent itself.

💡 **Core Insight:** "Walk through the workflow first. Let the agent fail. Correct it. Then codify it." — Ras Mic

A skill built this way isn't your instructions to the agent — it's a DISCOVERED procedure that you and the agent developed together through real execution and correction.

The 5-Phase Recursive Skill Building Process

PHASE 1: MANUAL WALKTHROUGH

Do the workflow MANUALLY with the agent, step by step.
Don't ask for the whole workflow at once.
Guide each step yourself.

Example (Sponsor Vetting):

You: "Here's a sponsor email. Check their Twitter."

AI: [checks Twitter] → Result

You: "Now check their YouTube channel."

AI: [checks YouTube] → Result

You: "Now check Trustpilot."

AI: [checks Trustpilot] → Result

You: "Based on these 3 sources, recommend accept/reject."

AI: [makes recommendation]

PHASE 2: CAPTURE THE WORKFLOW

After completing the manual walkthrough:

You: "Create a skill.md file that captures exactly
what you just did, step by step."

The agent writes its OWN instructions based on what
it actually did – not what you THINK it should do.

This is the key difference from writing skills manually.

PHASE 3: FIRST LIVE TEST

Run the new skill on a real task.

Don't intervene – let it attempt the full workflow.

"It WILL fail. That's expected. That's good."

Note WHERE it fails and HOW.

PHASE 4: FAILURE ANALYSIS → SKILL IMPROVEMENT

When it fails:

You: "Why didn't you check Trustpilot? The skill says to."

AI: "You're absolutely right. I should have."

→ Update the skill file with more explicit instructions

→ Add decision rules for edge cases you discovered

→ Add clarifying examples of what "good" looks like

PHASE 5: RECURSIVE REFINEMENT

Repeat Phase 3 & 4 for 3-5 iterations.

Each iteration, the skill gets more robust.

After 5 iterations, the skill runs "flawlessly."
"Genuinely think if anyone is going to sell skills,
there's going to be people who sell really well-defined
step-by-step skills." – Ras Mic

Why Failures Are Features, Not Bugs

This is the most psychologically important insight: **when your agent fails while following a skill, this is the most valuable learning moment you can have.**

Quote from Video: "When it messes up, you thank God. You don't complain — because it messed up and it told you what was missing from the skill. Every failure teaches you what the skill needs to handle that case."

Each failure reveals a gap — an assumption you made, an edge case you didn't consider, an ambiguous instruction the model interpreted differently than you intended. Fixing these gaps is exactly how skills become robust.

The Skill Development Timeline

Iteration	Expected State of Skill	Your Action
Manual walkthrough	No skill file yet	Guide every step, observe
After Phase 2	Rough skill file (v0.1)	First live test
After 1st failure	v0.2 — missing edge cases	Add explicit rules
After 2nd failure	v0.3 — ambiguous instructions	Clarify with examples
After 3rd run	v0.4 — mostly working	Test edge cases
After 4th–5th runs	v1.0 — production ready	Deploy with confidence

 **Pro Tip:** Version control your skill files with Git. Each iteration is a commit. When a new model version comes out, you can test your skills against it systematically and see where the new model changes behavior.

Common Skill-Building Mistakes to Avoid

- **Writing skills from scratch without running them first** — You'll miss the edge cases you don't know exist yet
- **Making skills too broad** — "Marketing skill" is too vague; "LinkedIn post from YouTube transcript" is specific enough
- **Not updating skills after failures** — The skill file should be a living document
- **Skipping the manual walkthrough phase** — You need to understand the workflow yourself before codifying it
- **Over-engineering on the first attempt** — Start minimal, add complexity as you discover the need

8. Real-World Skill Deep Dives

Case Study 1: The Sponsor Vetting Skill

This is the first detailed example in the video. Ras Mic builds an OpenClaude agent specifically to vet sponsorship emails for his podcast.

THE PROBLEM:

Ras Mic gets many sponsorship emails weekly.
Manually vetting each one: ~30-60 minutes of research
Total weekly overhead: potentially 5-10 hours

First attempt WITHOUT a proper skill:
Agent assessed every single sponsor as "legit" –
zero rejections. It was just agreeable, not analytical.

THE FAILED FIRST ATTEMPT (lesson learned):

Instruction: "Vet this sponsor and tell me if
they're worth it"

Problem: No criteria! The agent had no definition of
"worth it" so it defaulted to being agreeable.
This is a context gap, not a model failure.

THE REFINED SKILL (after manual walkthrough):

Required checks:

- ✓ Twitter/X presence and quality
- ✓ YouTube channel (if applicable)

- ✓ Trustpilot reviews
- ✓ Crunchbase funding check
- ✓ "Have they raised money?" (proxy for legitimacy)

REJECTION RULE (the key addition):

"If 2 or more of these checks don't exist or are
in poor standing → automatic rejection"

OUTPUT:

Google Sheets entry: [company, verdict, confidence, reason]

Values: YES / NO / CONTACT

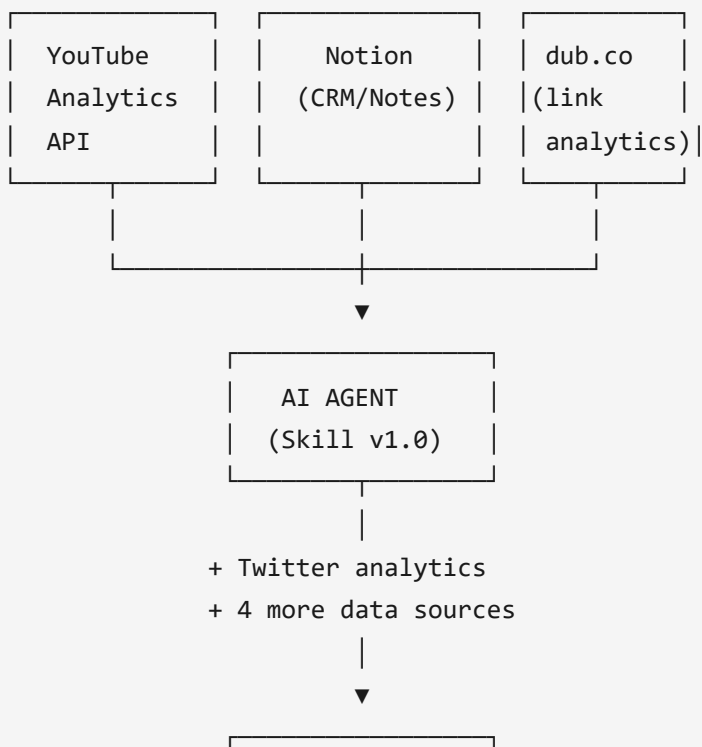
The agent now runs autonomously: Ras forwards sponsor emails to the agent's dedicated inbox → agent checks every 15 minutes → researches → logs verdict. A task that took hours is now fully automated with high accuracy.

Case Study 2: The YouTube Analytics Report Skill

This is the most technically complex example — it demonstrates what's possible when you build a skill that coordinates multiple data sources.

THE YOUTUBE ANALYTICS SKILL:

DATA SOURCES (8 total):



COMPREHENSIVE
REPORT
(10 min exec)

Key insight: "There's no way you're going to one-prompt this and have the agent do it. But every time I tell it to do that work, it takes 10 minutes and executes it flawlessly. Why? I went through FIVE loops of recursively building this skill."

This is a critical point about complexity management: tasks that seem impossible to automate with a single prompt become reliably automatable when broken into a well-tested skill procedure.

Case Study 3: The Code Structure Skill

This skill defines how generated code should be organized and presented for easy human review. It's 116 lines and 872 tokens.

CODE STRUCTURE SKILL METADATA:

name: code-structure
description: Use when multiple workflows duplicate
the same operational logic – when
deciding that [technical criteria].

WHY THIS WORKS:

The agent sees this description and knows:
"If I'm generating code and there's duplication,
load and follow this skill."

Without the skill: output is inconsistent,
hard to review, different each time.

With the skill: output follows a consistent
structure that Ras Mic finds easy to review,
every single time.

Token Efficiency Example: Without skills, this 116-line, 872-token document would be in the agent.md, loading on EVERY request regardless of relevance. With skills, those 872 tokens are only loaded when the agent is doing code-generation work — maybe 20% of sessions. That's an 80% reduction in wasted context tokens for this single skill.

9. Sub-Agents vs. Single Agents: When to Scale

One of the most tempting traps in AI agent building is over-engineering with multi-agent architectures before you need them. The video provides clear guidance on this.

💡 **The Core Principle:** "Scale for productivity, not for what looks cool." — Ras Mic Build the simplest system that solves the problem. Only add complexity (sub-agents, orchestrators) when you hit real bottlenecks.

Single Agent vs. Multi-Agent Architecture

SINGLE AGENT:

User → [Single Agent + Skills + Tools] → Output

- ✓ Simple to build, debug, maintain
- ✓ No coordination overhead
- ✓ Easier to understand what went wrong
- ✓ Lower cost (one LLM call chain)
- ✗ One context window limitation
- ✗ Can get confused with too many responsibilities

MULTI-AGENT (Orchestrator + Sub-Agents):

User → [ORCHESTRATOR AGENT]



[Research] [Writing] [Publishing]
Sub-Agent Sub-Agent Sub-Agent

- ✓ Each agent specializes in one area
 - ✓ Can parallelize (research + writing simultaneously)
 - ✓ Cleaner separation of concerns
 - ✓ Each agent has a focused, clean context
 - ✗ Complex coordination
 - ✗ Harder to debug (which agent failed?)
 - ✗ Higher cost (multiple LLM call chains)
-

The Decision Framework: When to Add Sub-Agents

Scenario	Use Single Agent	Use Sub-Agents
Task complexity	Linear, step-by-step workflow	Multiple independent parallel tasks
Specialization needed	One domain of knowledge	3+ very different domains
Context window pressure	Task fits in one context window	Task requires more than one context window
Team structure	Solo operator	Simulating a team with distinct roles
Error isolation	Errors are easy to trace	Need to isolate failures by function

💡 **Ras Mic's Rule:** "There's a perfect time to use sub-agents. I use sub-agents a lot. But I call it 'scaling for productivity, not for what looks cool.'"

Start with a single agent. Hit a real wall. Then add a sub-agent for EXACTLY the part that's bottlenecked. Don't design a 5-agent system upfront for a problem that one well-skilled agent could solve.

The "Build from Scratch" Principle

Ras Mic makes an important point about tools like Paperclip (pre-built multi-agent systems): they look impressive and they ARE impressive — but building your OWN system from scratch gives you something much more valuable: **deep understanding of what your specific workflow needs**.

When you build from scratch:

- You know exactly why each component exists
- You can optimize for YOUR specific use case
- You understand failure modes and can fix them
- The system matches your mental model perfectly
- You can sell or share it because you understand it completely

10. The AI Workflow Ladder: From Simple to Complex

A critical insight for anyone building with AI: there's a natural progression from simple to complex systems, and most people jump too far up the ladder too quickly.

THE AI WORKFLOW LADDER:

RUNG 5: MULTI-AGENT ORCHESTRATION

Multiple specialized agents working in parallel
Orchestrator routes work to the right specialist
Example: Full autonomous business operations

RUNG 4: SINGLE AGENT WITH RICH SKILL LIBRARY

One agent, 10+ custom skills, multiple tool integrations
Can handle virtually any workflow in your domain
Example: Ras Mic's YouTube ops agent

RUNG 3: SINGLE AGENT WITH BASIC SKILLS

One agent, 2-3 skills, 1-2 tool integrations
Handles your most important workflows reliably
Example: Sponsor vetting + one other workflow

RUNG 2: PROMPTED WORKFLOW (No Skills Yet)

Detailed prompts for specific tasks
No persistent skills, manual step-by-step
Example: Manually walk through sponsor vetting each time

RUNG 1: BASIC CHAT + INSTRUCTIONS

Just typing prompts and getting responses
No system, no workflow, no persistence
Example: "Hey Claude, help me with this email"

MOST PEOPLE MISTAKE: Jump from Rung 1 to trying Rung 5

CORRECT PATH: Build Rung 2 → Rung 3 → Rung 4 → Rung 5

The "Walk the Workflow" First Principle

This is the behavioral pattern Ras Mic recommends for moving up the ladder. Before you automate anything:

1. **Do it manually with the agent** — Step through every action one by one with explicit guidance
2. **Observe where friction occurs** — What does the agent get wrong? What needs clarification?
3. **Identify the decision points** — What judgment calls need to be made? Can they be made explicit?
4. **Codify into a skill** — Once you've walked it 2-3 times and it works, write the skill
5. **Test autonomously** — Let the skill run end-to-end
6. **Iterate on failure** — Each failure is a skill improvement opportunity

Understanding Why Agents "Fail" (And What to Do)

Most agent failures fall into one of these categories:

Failure Type	Root Cause	Fix
"It didn't do what I wanted"	Ambiguous goal specification	Be more explicit about desired outcome
"It skipped a step"	Implicit assumptions in skill file	Make every step explicit in skill
"It made up information"	No grounding data available	Provide data sources in the workflow
"It did it differently each time"	No output format specified	Add explicit format requirements to skill
"It gave me generic answers"	Missing personal/business context	Add your specific context to system prompt
"It got confused halfway through"	Task too long for one context window	Break into sub-skills or sub-agents

11. The Skill Marketplace: A Coming Economy

One of the most forward-looking predictions in the video: **well-built AI skills will become a marketable, sellable asset**. This is an emerging opportunity most people haven't recognized yet.

Quote from Video: "I genuinely think if anyone is going to — if a skills marketplace is going to be a thing — there are going to be people who sell skills: really well-defined, step-by-step skills. Because people are just creating them without having built out the workflow with the agent first."

Why Skills Have Intrinsic Value

A high-quality skill file has real economic value because:

- **Time to build** — 5 iterations of refinement represents real expertise and hours of work
- **Domain knowledge encoded** — The skill captures judgment calls that took experience to develop
- **Failure-tested** — It's been run against real scenarios and refined for robustness
- **Edge cases handled** — Decision logic for unusual situations is explicitly encoded
- **Plug-and-play** — A buyer can immediately get 80% of the value without building from scratch

The Skill Economy Model

EMERGING SKILL ECONOMY:

SKILL CREATORS (Supply Side):

Experts in specific domains build and refine skills
Through 5+ iterations of real-world testing

Example creators:

- Marketing consultant → LinkedIn content system
- Financial analyst → Investment research skill
- Sales rep → Lead qualification skill
- Developer → Code review skill

SKILL BUYERS (Demand Side):

Non-experts who want the capability without building it
Businesses setting up new AI agents

MARKETPLACE MECHANICS:

Price determined by: domain specificity, # iterations,
quality of documentation, failure edge cases covered

POTENTIAL PLATFORMS:

Similar to: Shopify app store, WordPress plugins,
or the emerging agent/tool marketplaces

Building Skills With Market Value in Mind

If you want to eventually sell skills, build them with these characteristics:

Characteristic	Why It Matters for Sales
Clear, specific name	Buyers can immediately understand what it does
Precise description	Buyers can determine fit without trying it
Version history documented	Shows maturity and refinement
Input/output examples included	Reduces buyer uncertainty
Model compatibility noted	Buyers know which LLMs it works with
Known limitations documented	Builds trust, reduces support requests

12. The Permanent Underclass Warning (And What It Actually Means)

The video touches on a concept spreading through AI discourse: the "permanent underclass" — people who don't adapt to AI tools and get left behind economically.

⚠️ **What Ras Mic Actually Says:** He pushes back on the term "permanent underclass" as unnecessarily terrifying. The real message is simpler: people who know how to effectively build and use AI agents will have a significant productivity and economic advantage over those who don't. This gap is growing, but it's not permanent — the tools are accessible to everyone.

The Real Skills Gap

THE CURRENT AI SKILLS DISTRIBUTION:

Level 4 – BUILDERS (5-10%)

"Build and sell AI skills, workflows, and agents"

Understand context windows, skills, sub-agents

Build from scratch for specific use cases

Economic advantage: VERY HIGH

Level 3 – POWER USERS (15-20%)

"Use AI deeply with custom workflows and some skills"

Have system prompts and a few skills

Regular AI-augmented work

Economic advantage: HIGH

Level 2 – CASUAL USERS (40%)

"Use AI occasionally for specific tasks"

Copy-paste prompts from Twitter

Use ChatGPT like a search engine

Economic advantage: MODERATE

Level 1 – NON-USERS (35-40%)

"Rarely or never use AI tools"

Don't use AI in workflow

Economic advantage: NONE

The message is not fear-mongering — it's an invitation. The tools to become a Level 3 or Level 4 operator are free or cheap and available to everyone. The barrier is knowledge, not access.

13. Building Your Personal AI System: The 30-Day Action Plan

Here is a concrete, step-by-step plan to apply everything in this tutorial, structured as a 30-day build program.

Week 1: Foundation (Days 1–7)

Day 1-2: Audit your current AI usage

- List the top 10 tasks you do weekly where you use or could use AI
- Estimate time spent on each
- Identify which are repetitive workflows (skill candidates) vs. one-offs

Day 3-4: Set up your agent environment

- Choose your primary agent (Claude.ai / Claude Code / OpenAgent)
- Write a minimal system prompt (under 300 tokens) for your primary use case
- Test it with 5 different real tasks

Day 5-7: Strip down your instructions

- If you have an existing agent.md or system prompt, audit it
- Remove any instructions that state general knowledge
- Keep only: your specific preferences, rules unique to your workflow, output format specs

Week 2: First Skill (Days 8–14)

Day 8-9: Choose your first skill workflow

- Pick the MOST repetitive task from your Week 1 audit
- This should be something you do 3+ times per week
- It should have a clear, repeatable structure

Day 10-11: Manual walkthrough (Phase 1)

- Do the workflow manually with the agent, step by step
- Guide every single action yourself
- Note every place where you had to correct or guide

Day 12: Create the skill file (Phase 2)

- Ask the agent: "Create a skill.md file that captures exactly what we just did"
- Review it — add/remove based on your observations from Day 10-11
- Write clear metadata: name + description

Day 13-14: First live test + iteration (Phase 3-4)

- Run the skill end-to-end on a real task
- When it fails (it will), note exactly where
- Update the skill file with explicit instructions for the failure case
- Run again

Week 3: Expand the Library (Days 15–21)

Build 2 more skills using the same methodology

- One skill for a communication workflow (email, outreach)
- One skill for a research/analysis workflow
- Run each through 3 iterations minimum

Integrate tool access

- Connect your agent to at least one external data source (Google Sheets, email, web search)
- Build a skill that uses this integration
- Test the full end-to-end automated workflow

Week 4: System Polish (Days 22–30)

Skill refinement

- Run all 3 skills 5+ times each
- Document edge cases and how skills handle them
- Consider sharing or selling your best skill

System evaluation

- Measure: How much time did AI save you this week?
- Identify the next 3 workflows to build skills for
- Plan for Month 2 expansion

Quick Reference: The Core Principles

#	Principle	Application
1	Models are already good	Stop teaching the model — start building context
2	Skills > Agent.md	Use on-demand loading instead of always-on bloat
3	Walk before you codify	Manual walkthrough first, skill file second
4	Failures are features	Each failure tells you what the skill needs
5	Progressive disclosure	Only load what's relevant to the current task
6	Scale for productivity	Add sub-agents only when you hit real bottlenecks
7	5 iterations minimum	A skill isn't production-ready until it's been tested 5x
8	Context = output quality	Better context structure = dramatically better results

14. Glossary of Key Terms

Term	Definition
AI Agent	A goal-oriented AI system that autonomously decides which tools to use and what actions to take to achieve an objective
Context Window	The total amount of text (measured in tokens) that an AI model can "see" and reason about in a single interaction
System Prompt	Permanent instructions given to an AI agent before any conversation begins; defines role, behavior, and rules
Agent.md / CLAUDE.md	A markdown file that's automatically loaded into an AI agent's context at the start of every session
Skill	A markdown file containing step-by-step procedures for a specific workflow, loaded on-demand only when relevant
Progressive Disclosure	The pattern where a skill's full content is only loaded after the agent determines it's relevant (via reading just the name + description)
Recursive Skill Building	The methodology of building skills through iterative manual walkthrough → codification → live testing → failure analysis → improvement

Sub-Agent	A specialized AI agent that handles one specific domain or function, coordinated by an orchestrator agent
Orchestrator	A high-level agent that receives complex goals and routes sub-tasks to appropriate specialized sub-agents
Token	The basic unit of text that AI models process; roughly 0.75 words or 4 characters in English
LLM	Large Language Model — the underlying AI model (Claude, GPT-4, Gemini) that powers agents
ReAct Loop	Reasoning + Acting — the iterative think-act-observe loop that agents use to accomplish goals

About This Document

This tutorial was created by analyzing the full transcript and insights from the YouTube video "**How AI Agents & Claude Skills Work (Clearly Explained)**" by Greg Isenberg featuring Ras Mic.

Video URL: https://www.youtube.com/watch?v=S_oN3vlzpMw

Channel: Greg Isenberg (644K subscribers)

Duration: 35:25

Views: 454K+

Next Steps:

- Watch the original video for the full conversational context and examples
- Follow Ras Mic for more in-depth AI agent methodology
- Start with your first skill today — pick the most repetitive task you have
- Share this document with anyone building AI agent workflows